# Documentación Técnica — Sistema HFES

Rincón Guatemalteco — Punto de Venta Web

**Autor:** Héctor Espinoza **Versión:** 1.0.0 **Fecha:** Octubre 2026 **Repositorio:** `PF_Web`

## 1. Descripción General del Sistema

HFES (Héctor Espinoza Sistema) es una aplicación web de punto de venta desarrollada para el negocio **Rincón Guatemalteco**. El sistema permite gestionar productos, clientes, usuarios, ventas y cajas registradoras a través de una interfaz moderna con control de acceso basado en roles.

### 1.1 Objetivos del Sistema

- Registrar y controlar ventas en tiempo real desde cajas asignadas a cajeros.
- Gestionar el inventario de productos con descuento automático de stock al vender.
- Administrar usuarios con distintos niveles de acceso (roles).
- Generar y consultar facturas con su detalle de productos.
- Proporcionar una interfaz intuitiva separada por rol de usuario.

### 1.2 Alcance

| Módulo | Descripción |
|---|---|
| Autenticación | Login con validación de estado y redirección por rol |
| Roles | CRUD de roles del sistema |
| Categorías | CRUD de categorías de productos |
| Productos | CRUD con categoría y estado asignados |
| Clientes | CRUD de clientes registrados |
| Usuarios | CRUD con asignación de rol (solo Administrador) |
| Punto de Venta | Pantalla exclusiva para cajeros con carrito y cobro |
| Facturas | Consulta de ventas y detalles (Admin y Sub Admin) |
| Perfil | Vista de datos del usuario autenticado |

## 2. Stack Tecnológico

### 2.1 Backend

| Componente | Tecnología | Ver. |
|---|---|---|
| Lenguaje | Java | 21 |
| Framework | Spring Boot | 4.1.2 |
| Persistencia | Spring Data JPA | — |
| ORM | Hibernate | — |
| Base de Datos | MySQL | 8 |

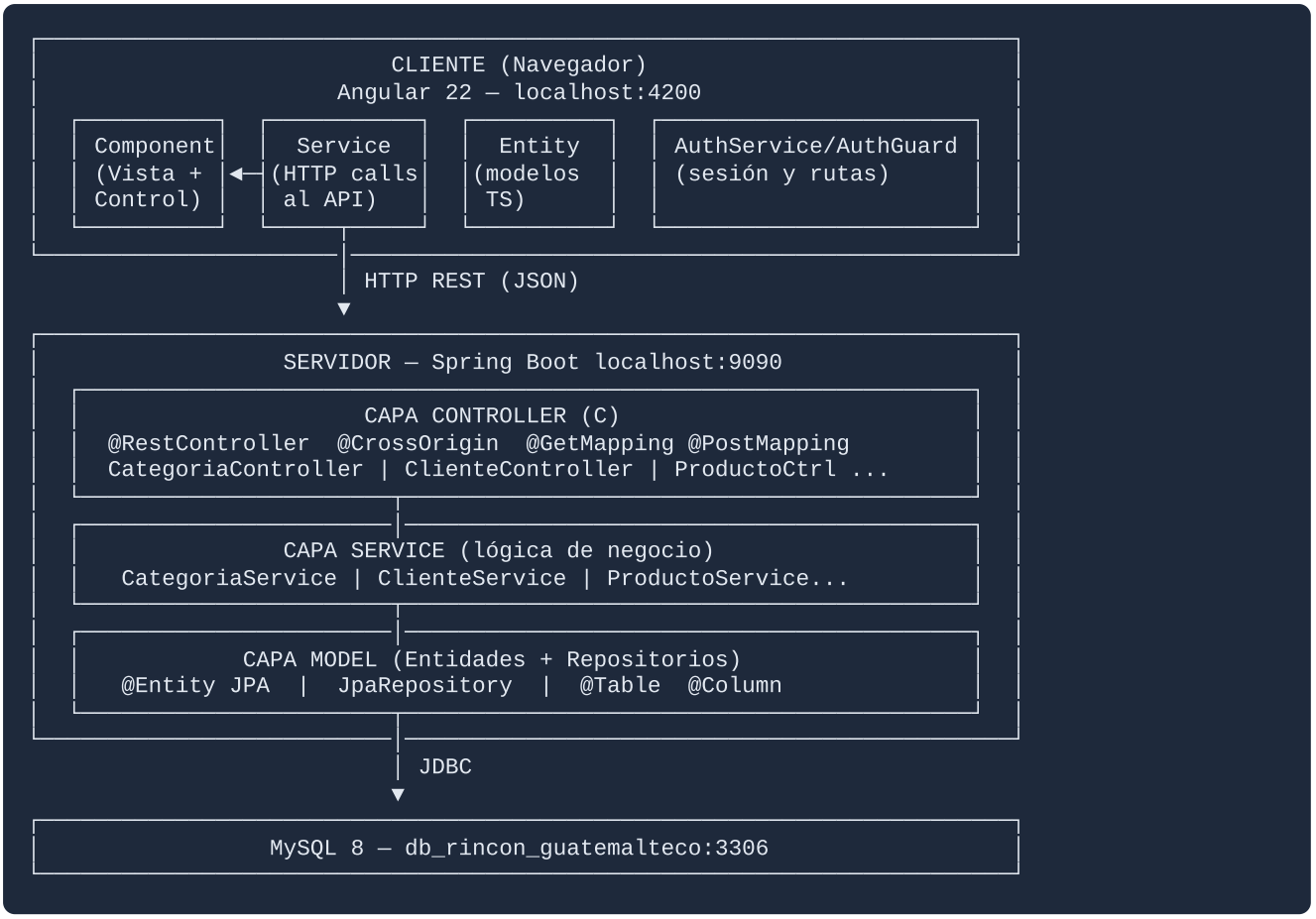
### 2.2 Frontend

| Componente | Tecnología | Ver. |
|---|---|---|
| Lenguaje | TypeScript | ~6.0 |
| Framework | Angular | 22.x |
| Estilos | CSS puro | — |
| Formularios | Angular FormsModule | — |
| HTTP | Angular HttpClient | — |

# 3. Arquitectura del Sistema

El sistema sigue el patrón **MVC (Modelo - Vista - Controlador)** tanto en el backend como en el frontend, adaptado a cada tecnología para garantizar la separación de responsabilidades y la escalabilidad del código.

## 3.1 Diagrama de Arquitectura General




## 3.2 Patrón MVC en el Backend (Spring Boot)

| Capa | Responsabilidad | Clases Principales |
|---|---|---|
| **Model** | Entidades JPA y Repositorios de acceso a datos | `Categoria`, `Producto`, `CategoriaRepository` ... |
| **View** | No aplica (API REST, la Vista se delega a Angular) | — |
| **Controller** | Recibe peticiones HTTP, llama al Service, retorna JSON | `CategoriaController`, `VentaController` ... |
| **Service** | Lógica de negocio, intermediario entre Controller y Repository | `CategoriaService`, `VentaService` ... |

## 3.3 Patrón MVC en el Frontend (Angular)

| Capa | Responsabilidad | Archivos / Estructura |
|---|---|---|
| **Model** | Interfaces y clases TS que mapean la respuesta JSON | `Entity/*.ts` |
| **View** | Templates HTML renderizados con directivas de Angular | `Component/*.html` |
| **Controller** | Lógica del componente, maneja el estado e invoca Services | `Component/*.ts` |

# 4. Estructura de Directorios

### 4.1 Backend — `hfes/`

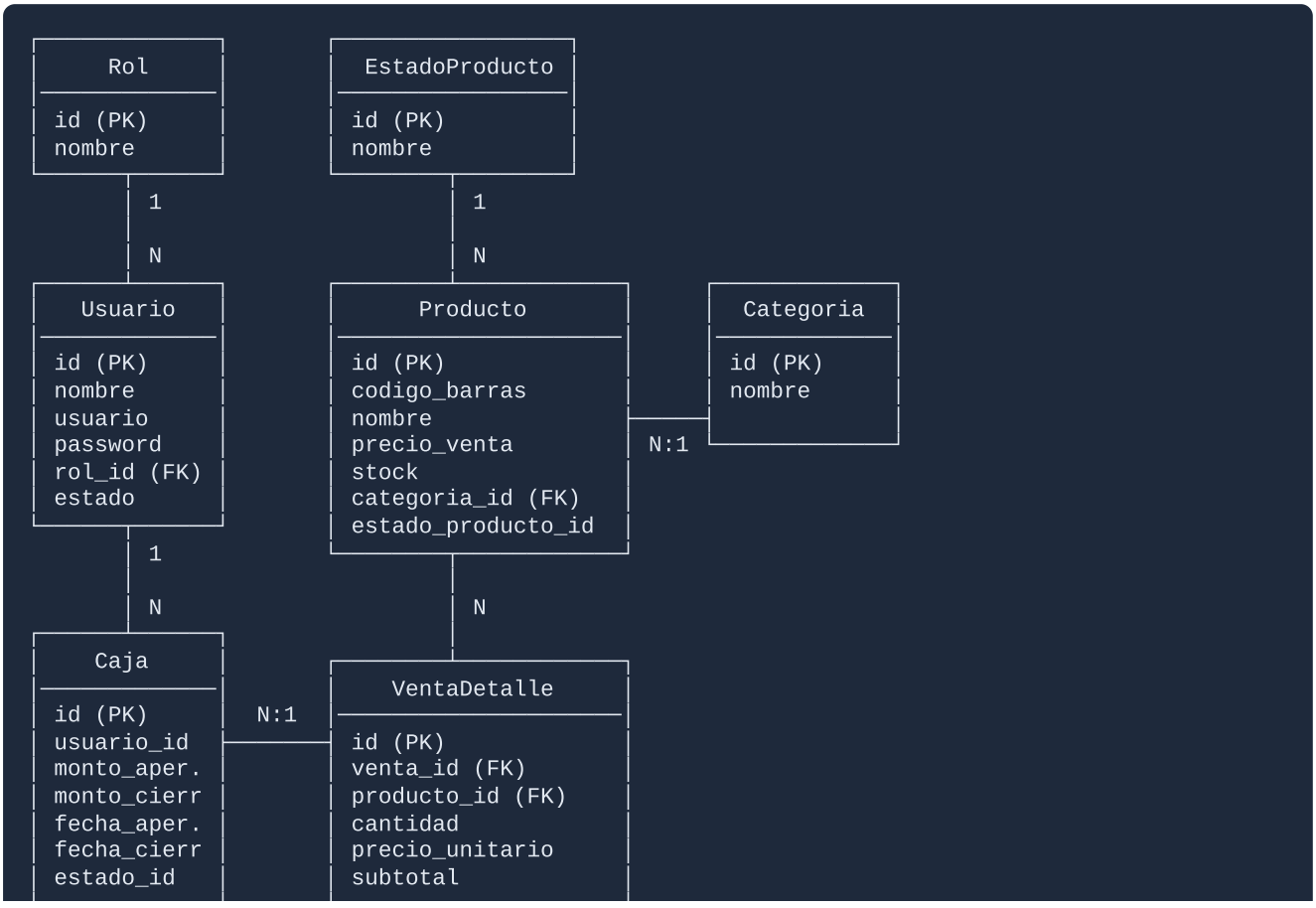
```
hfes/
└── src/main/java/.../hfes/
    ├── HfesApplication.java
    ├── Controller/
    │   ├── CajaController.java
    │   ├── CategoriaController.java
    │   ├── ClienteController.java
    │   ├── ProductoController.java
    │   ├── RolController.java
    │   ├── UsuarioController.java
    │   └── VentaController.java ...
    ├── Model/
    │   ├── Entity/
    │   │   ├── Caja.java
    │   │   ├── Categoria.java
    │   │   ├── Cliente.java
    │   │   ├── Producto.java
    │   │   └── Venta.java ...
    │   └── Repository/
    │       ├── CajaRepository.java
    │       └── ... (uno por entidad)
    └── Service/
        ├── CajaService.java
        └── ... (uno por entidad)
```

### 4.2 Frontend — `src/app/`

```
app/
├── app-module.ts
├── app-routing-module.ts
├── app.ts / app.html
├── Entity/
│   ├── Categoria.ts, Cliente.ts...
├── Service/
│   ├── auth.service.ts
│   ├── auth.guard.ts
│   ├── categoria.service.ts
│   └── producto.service.ts...
└── Component/
    ├── login/      ← Pantalla login
    ├── menu/       ← Sidebar
    ├── perfil/     ← Mi Perfil
    ├── categoria/  ← CRUD
    ├── cliente/    ← CRUD
    ├── producto/   ← CRUD
    ├── usuario/    ← CRUD (Admin)
    ├── facturas/   ← Consultas
    └── punto-venta/← Pantalla cobro
```

# 5. Modelo de Base de Datos

### 5.1 Diagrama Entidad-Relación



### 5.2 Detalle de Tablas y Columnas

| Tabla | Columnas Principales | Relaciones (FK) |
|---|---|---|
| `Rol` | id, nombre | — |
| `usuario` | id, nombre, usuario, password, rol_id, estado | rol_id → Rol |
| `categorias` | id, nombre | — |
| `estado_producto` | id, nombre | — |
| `productos` | id, codigo_barras, nombre, precio_venta, stock, categoria_id, estado_producoto_id | categoria_id → categorias estado_producoto_id → estado_producto |
| `clientes` | id, nit, nombre, direccion, telefono | — |
| `metodo_pago` | id, nombre | — |
| `estado_caja` | id, nombre | — |
| `cajas` | id, usuario_id, monto_apertura, monto_cierre, fecha_apertura, fecha_cierre, estado_caja_id | usuario_id → usuario estado_caja_id → estado_caja |
| `ventas` | id, caja_id, cliente_id, usuario_id, numero_factura, fecha_venta, total, metodo_pago_id | caja_id, cliente_id, usuario_id, metodo_pago_id |
| `venta_detalles` | id, venta_id, producto_id, cantidad, precio_unitario, subtotal | venta_id → ventas producto_id → productos |

## 6. API REST — Endpoints del Backend

**URL Base:** `http://localhost:9090`

### 6.1 Roles y Usuarios

| M. | Endpoint | Descripción |
|---|---|---|
| GET | `/roles` | Listar roles |
| POST | `/create_rol` | Crear rol |
| PUT | `/update_role/{id}` | Actualizar rol |
| GET | `/usuarios` | Listar usuarios |
| POST | `/create_usuario` | Crear usuario |
| PUT | `/update_usuario/{id}` | Actualizar usuario |

### 6.2 Categorías y Productos

| M. | Endpoint | Descripción |
|---|---|---|
| GET | `/categorias` | Listar categorías |
| POST | `/create_categoria` | Crear categoría |
| PUT | `/update_categoria/{id}` | Actualizar cat. |
| GET | `/productos` | Listar productos |
| POST | `/create_producto` | Crear producto |
| PUT | `/update_producto/{id}` | Actualiza incl. stock |

### 6.3 Ventas y Cajas

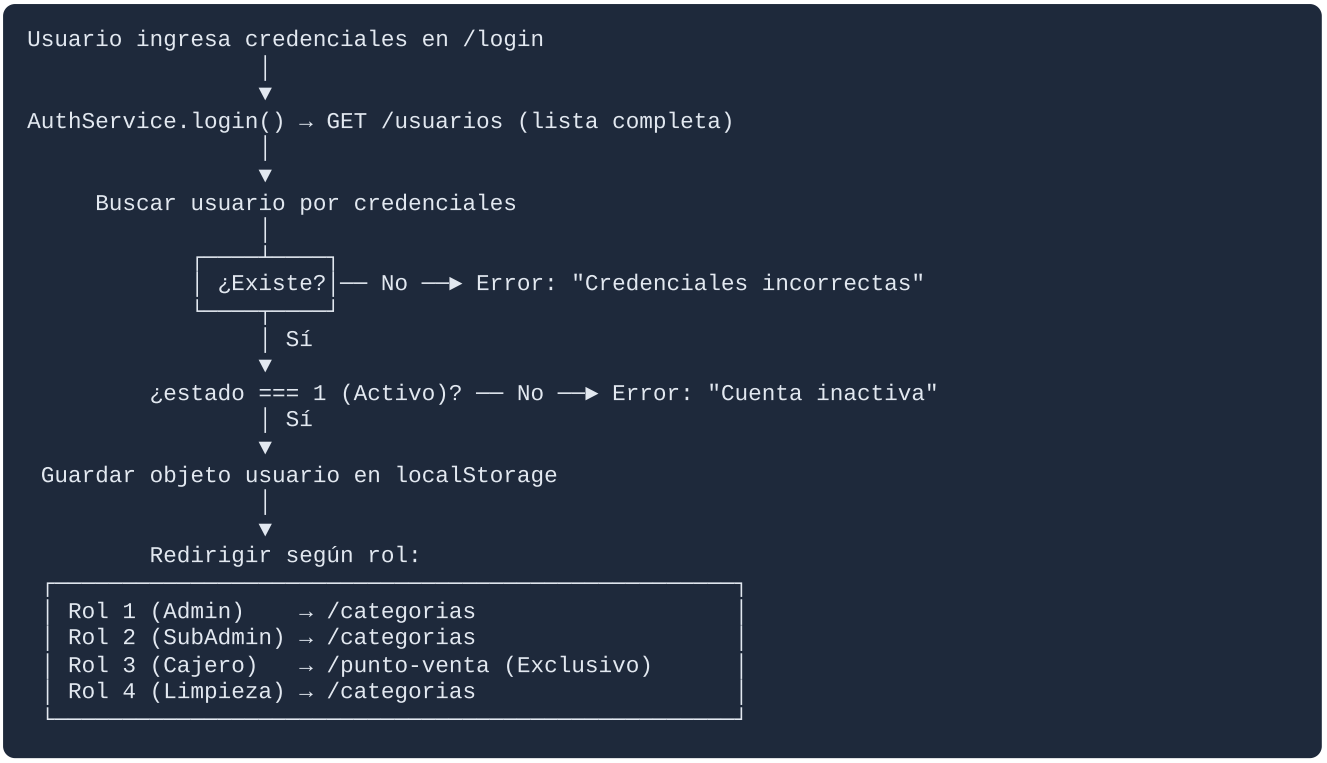
| M. | Endpoint | Descripción |
|---|---|---|
| GET | `/ventas` | Listar ventas |
| POST | `/create_venta` | Crear |

### 6.4 Clientes y Catálogos

| M. | Endpoint | Descripción |
|---|---|---|
| GET | `/clientes` | Listar clientes |
| POST | `/create_cliente` | Crear cliente |

# 7. Flujo de Autenticación y Control de Acceso

## 7.1 Flujo de Login y Redirección

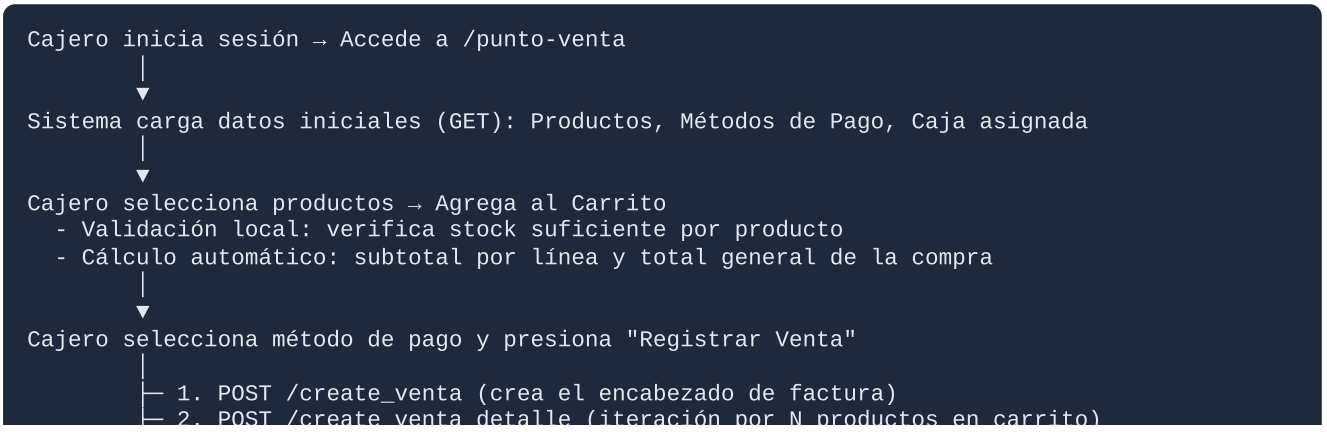



## 7.2 Matriz de Control de Acceso por Rol

| Módulo | Admin (1) | Sub Admin (2) | Cajero (3) | Limpieza (4) |
|---|---|---|---|---|
| Categorías, Clientes, Productos, Roles | ✅ | ✅ | ❌ | ❌ |
| Usuarios (Creación y Asignación) | ✅ | ❌ | ❌ | ❌ |
| Facturas (Historial) | ✅ | ✅ | ❌ | ❌ |
| Punto de Venta (Cobro) | ❌ | ❌ | ✅ | ❌ |
| Mi Perfil | ✅ | ✅ | ✅ | ✅ |

*Nota: El `AuthGuard` protege todas las rutas. Si no hay sesión en localStorage, redirige a `/login`.*

# 8. Flujo del Punto de Venta (POS)



# 9. Configuración y Despliegue

## 9.1 Requisitos Previos

| Herramienta | Versión mínima |
|---|---|
| Java JDK | 21 |
| Apache Maven | 3.9+ |
| MySQL Server | 8.0+ |
| Node.js & npm | 18+ / 9+ |
| Angular CLI | 22+ |

## 9.2 Variables y Puertos

| Elemento | Valor / Configuración |
|---|---|
| Puerto Backend | `9090` (application.properties) |
| Puerto Frontend | `4200` (Angular CLI) |
| URL API | `http://localhost:9090` |
| BD URL | `jdbc:mysql://localhost:3306/...` |
| BD Usuario/Pass | `root` / `root` |

**Comandos de Ejecución Rápida:**

```
# Base de Datos (MySQL)
CREATE DATABASE db_rincon_guatemalteco;

# Levantar Backend (Directorio hfes/)
mvn clean package
java -jar target/hfes-0.0.1-SNAPSHOT.jar

# Levantar Frontend (Directorio hfesfrontendpf/)
npm install
ng serve
```

# 10. Patrones y Decisiones de Diseño

- **ChangeDetectorRef:** Debido a que Angular 22 con el builder basado en esbuild ejecuta las respuestas HTTP fuera de NgZone, los componentes usan `detectChanges()` tras cada suscripción para actualizar el DOM correctamente.
- **Componentes Standalone:** Todos los componentes (excepto App) son *standalone*. Esto elimina la necesidad de declarar NgModules adicionales y permite importar `CommonModule`, `FormsModule` directamente donde se requieren.
- **Autenticación Simple:** Basada en `localStorage` del lado del cliente sin validación de token JWT en el backend. Adecuado para un entorno de red local cerrado (POS), pero requiere mejoras si se expone a internet.
- **Descuento de Stock:** Se orquesta desde el frontend llamando al endpoint `PUT` por cada ítem.

# 11. Posibles Mejoras Futuras

| Área | Descripción de la Mejora Propuesta |
|---|---|
| Seguridad | Implementar Spring Security + JWT para autenticación robusta y validación en API. |
| Transaccionalidad | Crear un endpoint dedicado en el backend que procese toda la venta y descuente el stock de forma atómica. |
| Reportes e Impresión | Módulo de reportes gráficos y generación de tickets de caja en formato PDF. |
| Paginación y Filtros | Implementar paginación del lado del servidor para tablas con grandes volúmenes de datos. |

## 12. Glosario Técnico

| Término | Definición |
|---|---|
| **HFES** | Héctor Fernando Espinoza Sistema — Siglas y nombre oficial del proyecto. |
| **MVC** | Model-View-Controller — Patrón arquitectónico utilizado para separar la lógica de negocio, la interfaz y el enrutamiento. |
| **JPA** | Java Persistence API — Estándar de Java que describe el manejo de datos relacionales (ORM). |
| **ORM** | Object-Relational Mapping — Técnica de programación para convertir datos entre sistemas de tipos incompatibles utilizando objetos. |
| **REST** | Representational State Transfer — Estilo arquitectónico de software para crear servicios web. |
| **CORS** | Cross-Origin Resource Sharing — Mecanismo que utiliza cabeceras HTTP para permitir a un user agent obtener permisos para acceder a recursos de un servidor en un origen distinto. |
| **CRUD** | Create, Read, Update, Delete — Las cuatro funciones básicas del almacenamiento persistente. |
| **DTO** | Data Transfer Object — Objeto utilizado para transferir datos entre capas de software. |
| **FK / PK** | Foreign Key (Clave Foránea) / Primary Key (Clave Primaria) en diseño de bases de datos relacionales. |
| **NgModule** | Clase marcada por el decorador @NgModule que configura el inyector y el compilador y agrupa componentes relacionados en Angular. |
| **Standalone** | Componentes, directivas o pipes de Angular que no requieren ser declarados en un NgModule. |
| **Guard** | Clase o función en Angular que devuelve un booleano o un Observable para controlar si el usuario puede activar o abandonar una ruta. |
| **Service** | Clase que proporciona una funcionalidad específica y bien definida. En Angular encapsula lógica HTTP; en Spring Boot encapsula lógica de negocio. |